

一、IVF（Inverted File Index）参数说明

1. nlist（倒排桶数量）

含义：
用于将特征空间划分成多少个聚类桶（cluster centroids）。

作用机制：

- 构建阶段使用 K-means 将所有向量聚类为 `nlist` 个中心。
- 每个中心对应一个倒排桶。
- 向量被分配到距离最近的中心。
- 查询时只搜索部分桶（由 `nprobe` 决定）。

影响：

增加 `nlist`：

- ✅ 桶划分更细，搜索更精确
- ✅ 提升精度上限（Recall 可更高）
- ❌ 索引训练时间上升（更多聚类中心）
- ❌ 索引体积和内存开销增加
- ❌ 小数据集时过大 `nlist` 会过拟合

推荐范围：

$\approx \sqrt{N} \sim 4\sqrt{N}$
(N 为数据库向量数量，例如 100k 向量 → 300~1200)

2. nprobe（查询桶数量）

含义：
查询时访问的倒排桶个数。

作用机制：

- 每次查询先找到最近的 `nprobe` 个聚类中心；
- 然后只在这些桶内搜索候选项；
- 取结果并排序。

影响：

增加 `nprobe`：

- ✅ Recall 提高（覆盖更多区域）
- ✅ 更接近精确搜索结果
- ❌ 查询时间线性增加
- ❌ CPU/GPU 负载更高

推荐范围：

1~64
(典型值：4, 8, 16, 32)

3. pq_m (PQ 子空间数, 可选)

含义：
用于 Product Quantization (向量压缩) 的分块数。

- 作用机制：
- 将特征向量分成 `m` 个子空间；
 - 每个子空间独立量化成离散码；
 - 查询时用查表计算近似距离。

影响：
增加 `pq_m`：
 精度提高 (更细分量化)
 内存和索引大小上升
 训练时间增加

推荐范围：
16、32、64 (视维度而定，一般维度/`m` $\approx$ 8~16)

4. pca_dim (PCA降维维度, 可选)

含义：
在索引前先使用 PCA 降维。

- 作用机制：
- 减少向量维度，压缩数据并去除噪声，提升搜索速度。

影响：
减少 `pca_dim`：
 训练/查询更快
 内存与索引更小
 Recall 降低 (信息损失)

推荐范围：
64、128、256
(取原维度的 1/2~1/4)

二、HNSW (Hierarchical Navigable Small World Graph) 参数说明

1. M (每个节点的邻居数)

含义：
图中每个节点平均连接的邻居数量。

作用机制：

- 控制图的稠密程度；
- 每个节点与 `M` 个最近的节点建立连接；
- 查询时从入口节点开始沿边遍历。

影响：

增加 `M`：

- ✅ 提升 Recall（图更稠密，更易找到近邻）
- ✅ 提升搜索稳定性
- ❌ 内存占用线性增加（每个节点更多边）
- ❌ 构建时间上升

推荐范围：

8~64（常用 16、32）

2. efConstruction（构建时候选集大小）

含义：

索引构建时，每个新节点在插入图中时考虑的候选邻居数量。

作用机制：

- 控制构图时的近似度；
- 大的 `efConstruction` 意味着更精确的连接。

影响：

增加 `efConstruction`：

- ✅ 构建出的图更准确（更高 Recall）
- ❌ 构建时间显著上升
- ❌ 内存占用略增

推荐范围：

100~400（取决于数据量和维度）

3. efSearch（查询时候选集大小）

含义：

搜索时维护的候选节点数量。

作用机制：

- 搜索从入口点开始，维持一个候选优先队列（大小为 `efSearch`）；
- 值越大，遍历越多节点，结果越接近真实最近邻。

影响：

增加 `efSearch`：

- ✅ Recall 提高
- ❌ 查询时间线性增加

推荐范围：

50~200（常用 64, 128）

Grid Search结果

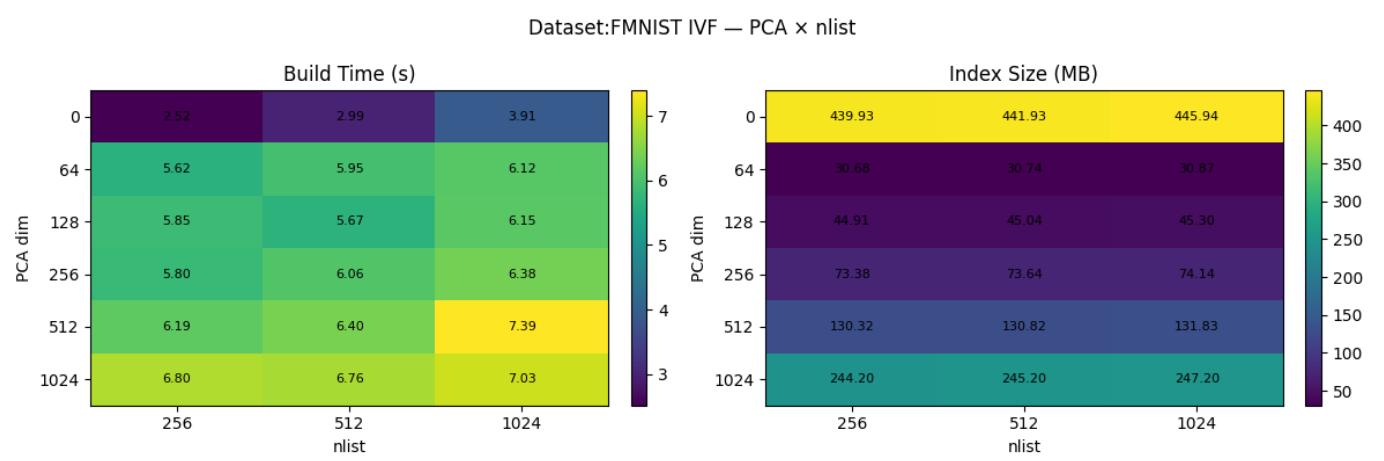
Benchmarks

评价Index性能的Benchmark如下

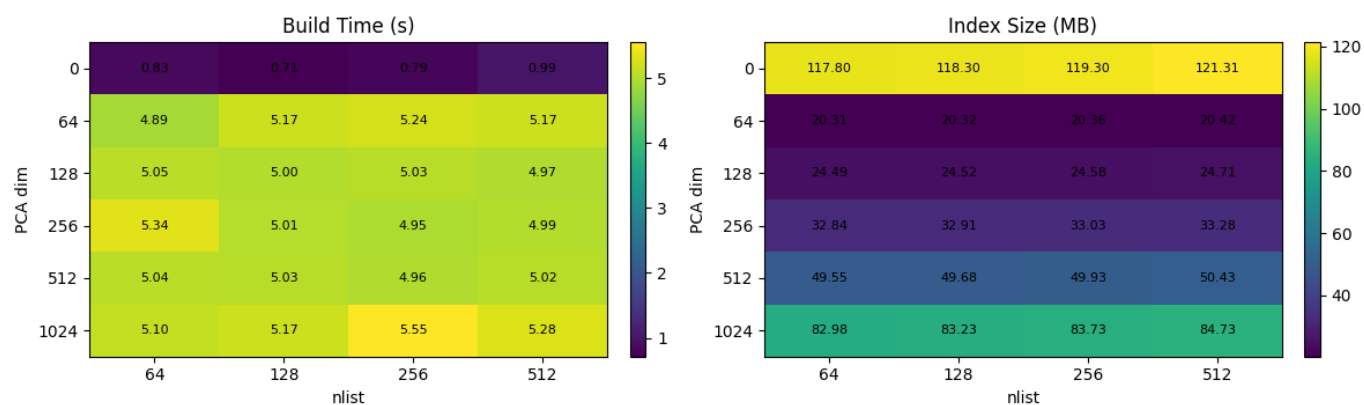
- Build Time: IVF, HNSW等模型需要预先训练以构建新的数据, 例如IVF分区和HNSW的图结构, 同时PCA压缩也需要预先transform所有Gallery中的向量。该数据越小越好, 即构筑Index的效率更高, 对于大尺度数据较为重要。
- Query Time: 即Latency, 查询给定新向量的neighbors时所用的时间。由于实验采用batch query, 即一次将测试集喂给index, 故测算了总时间后计算单个向量的平均时长作为估计值。越小越好
- Index Size: 即保存(持久化)后的index大小。由于实际上运用时FAISS Index必须存储在内存中否则I/O时间无法接受, 故相对较大的Index可能使大数量级数据下内存吃紧或不足, 难以执行大规模的分析。越小越好。
- Precision
 - On Label: 测算返回的K Neighbor中的类型判断正确率, 即(TP/TP+FN), 测算在压缩后是否能正确判断向量的label, 越高越好
 - On Vector: 测算返回的K Neighbors中符合Ground Truth (即由完全准确的L2FlatIndex返回的k neighbors) 的向量的正确率, 越高越好, 由于Ground Truth和实际QueryKNN时K值相同, 这里Precision@20=Recall@20。进而因为K值相同, 几乎等价地, Recall测算Groud Truth中的正确的KNeighbors向量的召回率, 即向量层面的匹配, 该值可能受因为PCA压缩或IVF等大幅影响, 不过本问题重点在Label层面的准确率, 该值仅供参考。

Recall on Label因为实际同label向量数量远大于K值, 故基本没有意义, 可以研究Precision。

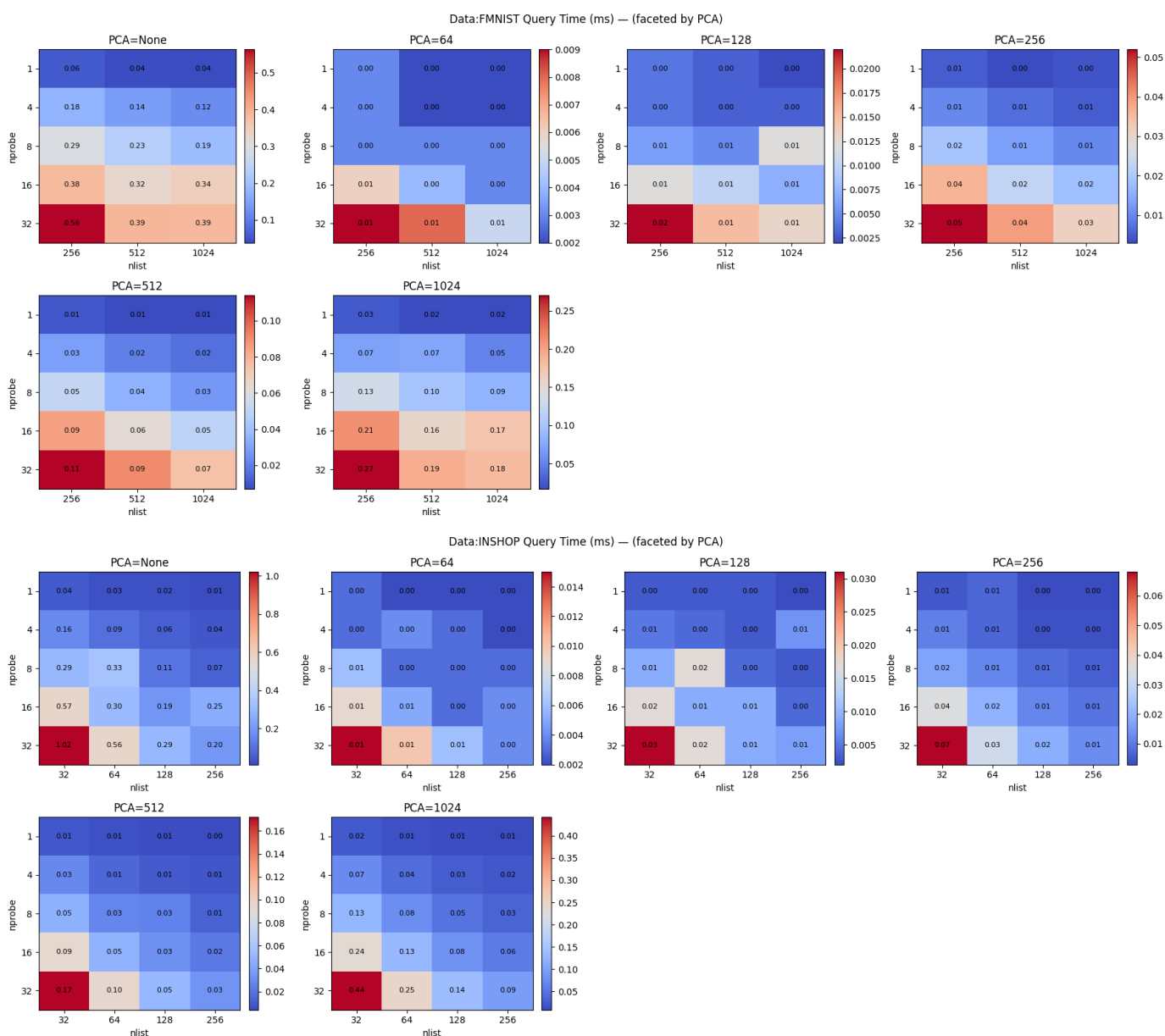
Result on Dataset: FMNIST(n=70k,2048dim) and INSHOP(n=15k, 2048dim)



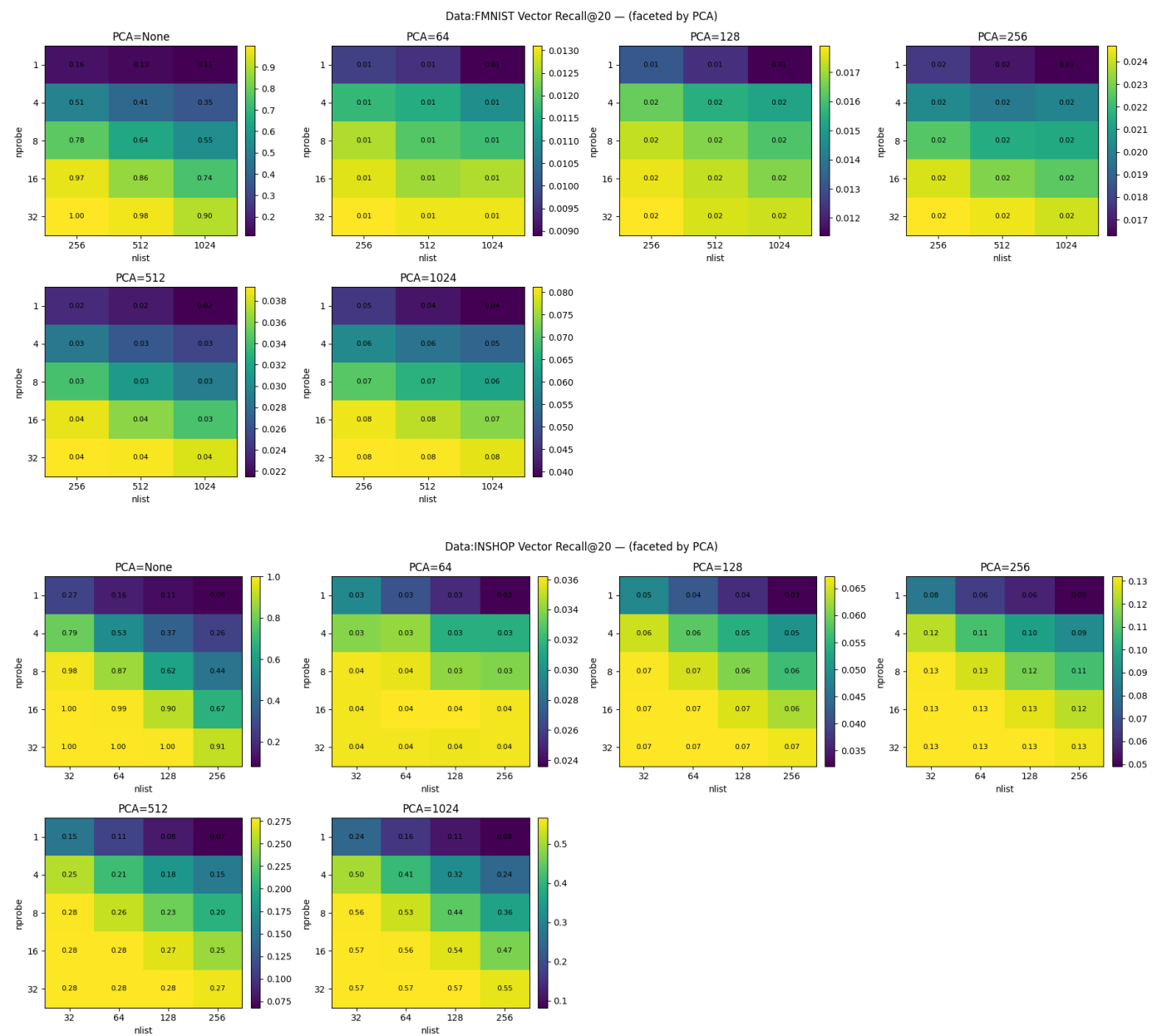
Dataset: INSHOP IVF — PCA × nlist



PCA=0即None，为不进行PCA压缩的结果。注意到PCA压缩和IVF构建均需要额外计算所以会增加index构建的时间，而nlist越多则kmeans计算时的centroid更多，计算更慢。最多达到两倍差距，tradeoff仍可接受。至于index大小，nlist对实际size影响不大，而PCA压缩后的维数则直接决定了每个向量实际保存的维数，进而对index大小产生线性关系的影响，故使用PCA Dim可以在 $O(n)$ 尺度上减少index需要的内存量。

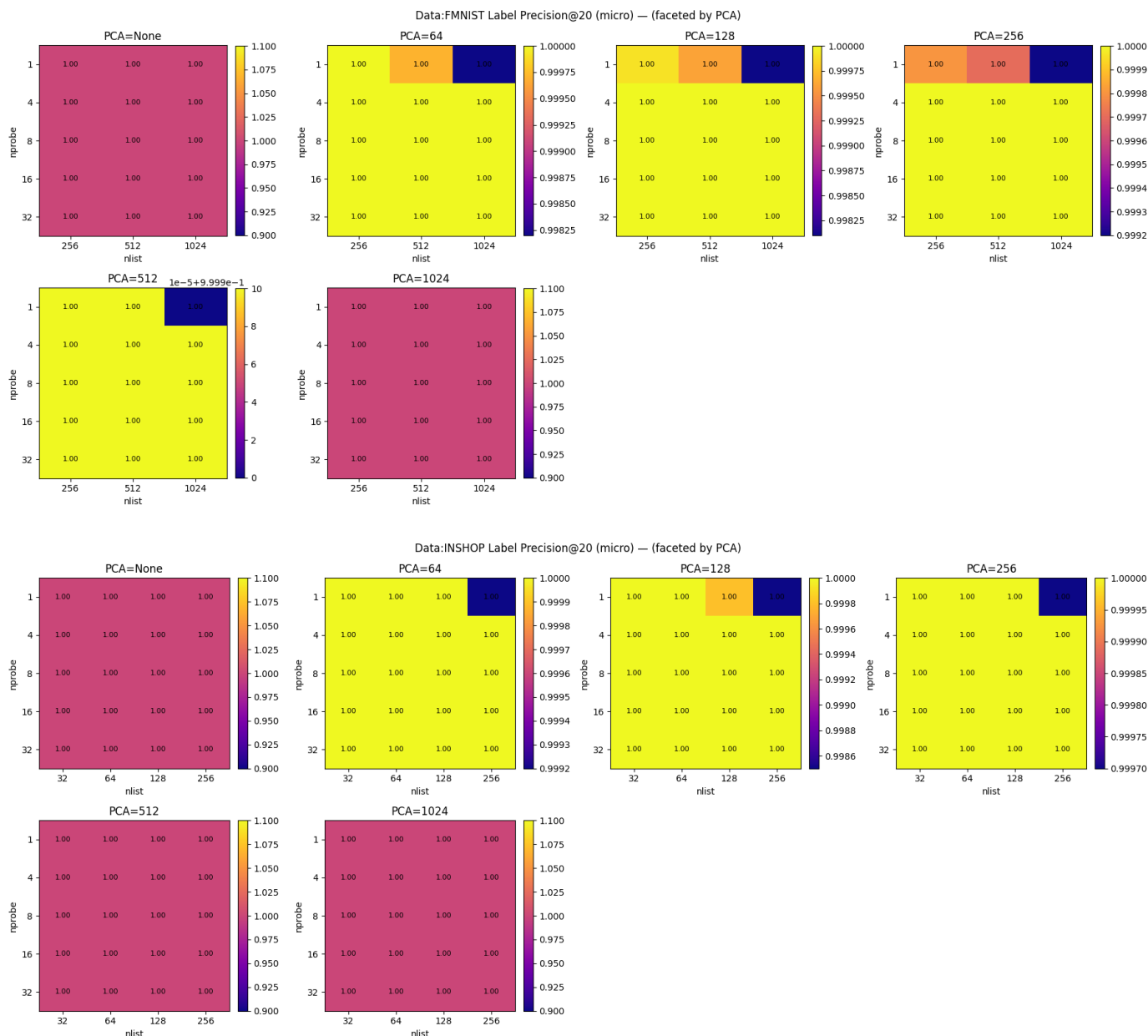


关于请求时间，由于nprob参数决定运行时进行搜索的cluster数量，其越大即会增加需要搜索的向量数量，继而对运行时间产生负面影响。同一nprob下更大的nlist会产生更细的切分进而减少搜索时间，但是注意到向下对角线方向，即nprob*nlist相等的方向随着nlist增加搜索时间会相对增加，虽然表面上进行比对向量接近，但更多的nlist，即cluster会增加I/O开销及有更多的稀疏向量需要比对，进而实际上增加了query time。



对于向量层面的Precision/Recall，PCA压缩会造成极大层面的影响。在无PCA压缩的情况下IVFIndex在大部分的nprob/nlist搭配下仍然能做到0.5~0.8左右的Recall，即能找到50%~80%其真正的最近邻。

即使对FMNIST（70k）数据集只压缩一半维度即压缩到PCA1024也使得Recall几乎下降到了0.1以下，基本可以认为完全失真，而在较小的数据集INSHOP（15k）下表现尚可，能做到0.57的Recall，我们可以认为L2归一化的源向量集在压缩后，更大的数据集由于更加稠密，因而在压缩后估计时更难命中真正的NN。



在在大幅压缩后两个数据集下基于Label匹配我们仍然能做到100%的Precision。考虑到原数据集相当大，如果只考虑类型的Label匹配时，大幅压缩也基本不影响precision。

Hyperparameters Optimization

我们设计一套简单的代码对超参数进行优化，对数据进行minmax standardization之后采用权重加权平均计算每种参数搭配的分，之后列出最好的3种组合。对权重设计了三种倾向，分别是速度型、平衡型、和准确度型。

由于该问题下label precision实际上相当容易保持在1，故我们给其较少的权重，注重vector层面的recall。

预先设计的三种参数搭配如下：

```
"speed": {
    "query_time_ms": ("min", 0.5),
    "build_time_s": ("min", 0.15),
    "index_size_mb": ("min", 0.10),
    "label_precision_at_k_micro": ("max", 0.2),
    "vector_recall_at_k": ("max", 0.05),
```

```

},
"balanced": {
    "query_time_ms": ("min", 0.2),
    "build_time_s": ("min", 0.15),
    "index_size_mb": ("min", 0.15),
    "label_precision_at_k_micro": ("max", 0.20),
    "vector_recall_at_k": ("max", 0.25),
},
"quality": {
    "label_precision_at_k_micro": ("max", 0.35),
    "vector_recall_at_k": ("max", 0.5),
    "query_time_ms": ("min", 0.10),
    "index_size_mb": ("min", 0.03),
    "build_time_s": ("min", 0.02),
},

```

加权平均投票给出的超参数组合如下

FMNIST数据集：

```

=== speed ===
  pca_dim  nlist  nprobe    score
0         64   256     16  0.865447
1         64   256      4  0.862864
2         64   256      8  0.862167

=== balanced ===
  pca_dim  nlist  nprobe    score
0         0   256     16  0.692398
1         0   512     32  0.692327
2         0   256      8  0.683172

=== quality ===
  pca_dim  nlist  nprobe    score
0         0   512     32  0.897138
1         0   256     16  0.891021
2         0   256     32  0.869351

```

INSHOP数据集：

```

=== speed ===
  pca_dim  nlist  nprobe    score
0         0    64      1  0.845644
1         0   256      1  0.844168
2         0    32      1  0.843661

=== balanced ===
  pca_dim  nlist  nprobe    score
0         0   128     32  0.780693
1         0   128     16  0.776672

```



```
2          0      32          8  0.775720
```

```
=== quality ===
```

	pca_dim	nlist	nprobe	score
0	0	128	32	0.939516
1	0	64	16	0.936451
2	0	32	8	0.932748

对于更小的INSHOP数据集能注意到由于BuildTime和QueryTime的Tradeoff更加明显，且PCA压缩对recall影响很大，所以即使在速度权重上升的情况下依然没有选择pca压缩。

注意到普遍的nlist（即centroid数量）取值在 $\sqrt{N} \sim 4\sqrt{N}$ 之间，即分别为264~1058及122~489左右，保证性能均衡下的nlist一般取了区间最小值即265及128。